1 Week 1 / KNN

1.1 The Very Basics

Learning: find a model f that maps from inputs to outputs given a labeled training dataset.
Inference: use a model f to predict an output y given a new example $\vec{x}$
A model is a function f mapping input $\vec{x}$ to output prediction y.
A model family is a set of models that share the same structure but differ in parameters (e.g. layers in a neural network
Accuracy is calculated as follows:

$$\frac{\sum_{i=1}^{N} \mathbb{I}\left[t^{(i)}=y^{(i)}\right]}{N}$$

A **parameter** is the internal variables of a model learned from data whilst training.
A **hyperparameter** are the settings chosen before training that controls how the learning process works.
Training vs. validation vs. test
Parametric models fix the parameter count. Non-parametric models don't. $\mathcal{O}(1)$ space complexity with respect to the feature count, and never $\Omega(N)$. Hyperparameters are out of this scope.

1.2 KNN

For 1NN: Find $t^{(c)}$ that corresponds to:

$$\vec{x}^{(c)} = \arg\min_{\vec{x}^{(i)} \in \text{training set}} \text{distance}\left(\vec{x}^{(i)}, \vec{x}\right)$$

For kNN, for the top k other datapoints closest to $\mathbf{x}$, choose the majority class.
Also good to normalize values:

$$x_{\text{norm}} = \frac{x-\mu}{\sigma}$$

Advantages:
- Simple
- No training
- Works for reg. and class.
- Adapts easily to new data

Limits:
- Bad w/ high dims
- Sensitive to scaling. Though can be normalized
- Slow for large datasets, time proportional to data
- Must store logs, proportional to data

If k is even, pick a tiebreaker. For this, you should make sure k is odd.

2 Decision Trees

Large trees overfit, small trees underfit. For inference, trace the tree.
Entropy

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$

Conditional Entropy

$$H(X|Y=y) = -\sum_{x \in X} p(x|Y=y) \log_2 p(x|Y=y)$$

Expected conditional entropy

$$H(X|Y) = -\sum_{y \in Y} p(y) \sum_{x \in X} p(x|y) \log_2 p(x|y)$$

Information gain

$$IG(X|Y) = H(X) - H(X|Y)$$

Properties
- Entropy nonnegative
- $H(Y|X) \leq H(Y)$
- Y, X are independent $\Rightarrow IG(Y|X) = 0$
- Full determinant rids it

2.1 Decision Tree Algorithm

Keep splitting until:
- All examples have the same label.
- When there are no features left.
- When a depth limit is reached. (hyperparameter)
- When the loss stops decreasing enough. (hyperparameter)

3 Linear / GD

Loss function measures how off once training example is.
Cost functions sum up all.

$$\mathcal{L}(y,t) = \tfrac{1}{2}(y-t)^2$$

$$\mathcal{E}(\mathbf{w}) = \tfrac{1}{N}\sum_{i=1}^{N} \mathcal{L}\left(y^{(i)}, t^{(i)}\right)$$

For the linear model $\mathbf{y} = W\mathbf{x}$, the **cost** function is:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}\left(\mathbf{w}^T\mathbf{x} - t^{(i)}\right)^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N}(X\mathbf{w} - \mathbf{t})^T(X\mathbf{w} - \mathbf{t})$$

3.1 Gradient of the Cost Function

$$\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{w}^T\mathbf{x}^{(i)} - t^{(i)}\right)x_j^{(i)}$$

$$\nabla_{\mathbf{w}}\mathcal{E}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{w}^T\mathbf{x}^{(i)} - t^{(i)}\right)\mathbf{x}^{(i)}$$

$$\nabla_{\mathbf{w}}\mathcal{E}(\mathbf{w}) = \frac{1}{N}X^T(X\mathbf{w} - \mathbf{t})$$

The regularizer:

$$\tfrac{\lambda}{2}\sum_{j=0}^{D} w_j^2$$

The gradient of the regularizer:

$$\frac{\partial}{\partial w_j}\left(\frac{\lambda}{2}\sum_{j=0}^{D} w_j^2\right) = \lambda w_j$$

$$\nabla_{\mathbf{w}}\left(\frac{\lambda}{2}\mathbf{w}^T\mathbf{w}\right) = \lambda\mathbf{w}$$

In the event the bias term is left in:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N}(X\mathbf{w} + b - \mathbf{t})^T(X\mathbf{w} + b - \mathbf{t})$$

$$\nabla_{\mathbf{w}}\mathcal{E}(\mathbf{w}) = \frac{1}{N}X^T(X\mathbf{w} + b - \mathbf{t})$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b - t^{(i)}\right)x_j^{(i)}$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial b} = \frac{1}{N}(X\mathbf{w} + b - \mathbf{t})$$

3.2 Stochastic GD

Batch GD is what GD normally does. It can be slow.
Stochastic GD just randomly picks a training example. It can be very noisy. Where i at random,

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$$

Mini-batch GD falls in-between.
Batch size can be tuned as a hyperparameter

3.3 Vectorizing Tips

Where X is $N \times D$ and all other variables can have their shape inferred:
Dot product

$$\sum_{i=1}^{N} u_i v_i = \mathbf{u}^T\mathbf{v}$$

Hadamard product

$$\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u})\,\mathbf{v} = \left(\mathbf{u}^T\text{diag}(\mathbf{v})\right)^T$$

Sum a vector

$$\mathbf{u}^T\mathbf{1} = \sum_{i=1}^{N} u_i$$

Summing a matrix on axis 1 - horizontal squish >−< (shape N)

$$X\mathbf{1}$$

Summing a matrix on axis 0 – vertical squish ↓ (shape D)

$$\mathbf{1}^T X$$

This is a column vector, and you may need to transpose it.
Row-wise multiplication

$$\text{diag}(\mathbf{u})\,X = \begin{bmatrix} - & \mathbf{x}^{(1)}u^{(1)} & - \\ - & \mathbf{x}^{(i)}u^{(i)} & - \\ - & \mathbf{x}^{(N)}u^{(N)} & - \end{bmatrix}$$

Column-wise multiplication

$$X \cdot \text{diag}(\mathbf{v}) = \begin{bmatrix} | & | & | \\ \mathbf{x}_1 v_1 & \mathbf{x}_2 v_2 & \mathbf{x}_3 v_3 \\ | & | & | \end{bmatrix}$$

Tip: in a chain of matrix multiplications, the **height** of the leftmost matrix will be the height of the output. E.g. for $X^T\mathbf{r}$, output is $D \times 1$.

$$\sum_{i=1}^{N} r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$$

4 Logistic Regression

Linear model is:

$$z = \mathbf{w}^T \mathbf{x}$$

Prediction is:

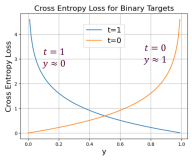
$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

For which the loss functions are:

- 0-1: $L_{0-1}(y,t) = \mathbb{I}[y \neq t]$
  - Hard to optimize
  - ∇ undef or 0
- $\hat{2}$: $L_{SE}(z,t) = \frac{1}{2}(z-t)^2$
  - Unbounded z
  - Large loss for very correct
- Logistic activation:
  $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $L_{SE}(y,t) = \frac{1}{2}(y-t)^2$
  - Prediction always [0,1]
  - ∇ small at large magnitude predictions
- Cross entropy: $\begin{cases} -\log(y) & t = 1 \\ -\log(1-y) & t = 0 \end{cases}$

Where cross entropy can be:

$$-t\log(y) - (1-t)\log(1-y)$$



4.1 Cross-Entropy Loss Equations

Linear model

$$z = \mathbf{w}^T \mathbf{x}$$

Classification

$$y = \sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$
$$\frac{\partial y}{\partial z} = y(1-y)$$

Loss

$$\mathcal{L}_{CE}(y, t) = \begin{matrix} -t\log(y) \\ -(1-t)\log(1-y) \end{matrix}$$

Gradient Descent Update

Where $\mathcal{E}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_{CE}\left(y^{(i)}, t^{(i)}\right)$

$$\begin{aligned} \frac{\partial \mathcal{L}_{CE}}{\partial w_j} &= \frac{\partial \mathcal{L}_{CE}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} \\ &= \left(\frac{t}{y} + \frac{1-t}{1-y}\right) \cdot y(1-y) \cdot x_j \\ &= (y-t)x_j \end{aligned}$$

Tip: re-use variables, don't sub all!

The gradient descent update for logistic regression is:

$$\begin{aligned} w_j &\leftarrow w_j - \alpha \frac{\partial \mathcal{E}}{\partial w_j} \\ &= w_j - \frac{\alpha}{N} \sum_{i=1}^{N} \left(y^{(i)} - t^{(i)}\right) x_j^{(i)} \end{aligned}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} X^T (X\mathbf{w} - \mathbf{t})$$